

Code Explanation

E2E Policy Services YUYU Extract Pipeline Explanation

The provided code is a Python script that serves as the main entry point for the E2E Policy Services YUYU extract pipeline. It utilizes the PySpark library to create a Spark session and perform various data processing tasks. The pipeline involves loading data from staging and lookup tables, enriching the data with lookups, applying business rule transformations, and writing the transformed data to a flat file output while updating the staging table.

Overview of the Code

This code is designed to execute a data processing pipeline that involves multiple steps, from loading data to writing the final output. It leverages PySpark for data processing and uses various custom classes for data loading, transformations, and writing.

Step 1: Creating a Spark Session

The first step involves creating a Spark session with the required configurations. This is achieved through the `create_spark_session` function, which initializes a Spark session with a specific application name and configurations.

```
def create_spark_session() -> SparkSession:
    config = Config()
    builder = SparkSession.builder.appName("E2E_YUYU_AddressChange_Extract")
    for key, value in config.get_spark_config().items():
        builder = builder.config(key, value)
    spark = builder.getOrCreate()
    spark.sparkContext.setLogLevel("WARN")
    return spark
```

Step 2: Initializing Pipeline Components

The main function initializes the Spark session and then creates instances of `DataLoader`, `YUYUTransformations`, and `DataWriter` classes, which are used for loading data, applying transformations, and writing the output, respectively.

```
spark = create_spark_session()
loader = DataLoader(spark)
transformer = YUYUTransformations()
writer = DataWriter()
```

Step 3: Loading Staging and Lookup Data

The pipeline loads data from a staging table and two lookup tables (`T_YUYU_CLNT` and `T_YUYUK_CLN`) using the `DataLoader` instance.

```
staging_df = loader.load_staging_data()
lookup_clnt_df = loader.load_lookup_yuyu_clnt()
lookup_cln_df = loader.load_lookup_yuyuk_cln()
```

Step 4: Enriching Data with Lookups

The loaded staging data is enriched with the lookup data using the `enrich\_with\_lookups` method of the `DataLoader` class.

```
enriched_df = loader.enrich_with_lookups(staging_df, lookup_clnt_df,
lookup_cln_df)
```

Step 5: Applying Business Rule Transformations

The enriched data is then subjected to various business rule transformations using the `apply\_all\_transformations` method of the `YUYUTransformations` class.

```
transformed_df = transformer.apply_all_transformations(enriched_df)
```

Step 6: Writing Flat File Output and Updating Staging Table

The transformed data is processed further to select columns required for the flat file output and staging table update. The `DataWriter` class is used to write the flat file output and update the staging table.

```
flat_file_df = transformer.select_flat_file_columns(transformed_df)
writer.write_flat_file_output(flat_file_df)

staging_update_df = transformer.select_staging_update_columns(transformed_df)
writer.update_staging_table(staging_update_df)
```